List 1: PyTorch & Training Fundamentals

Labs: Mon 5 Oct, Mon 12 Oct 2026 · Due: Sun 18 Oct 2026, 23:59 · 1 point

- For each task, show the requested numbers or plots and explain them in two to four sentences. The list is graded as a whole, in steps of 0.25, by how complete, correct and well explained your solution is.
- Data, code structure and names are up to you; defaults are given for when you have no preference. Keep the signatures of the four given functions, because later lists reuse them (check them with Materials, Appendix A).
- Background and the PyTorch functions you need for Task X.Y are in Materials X.Y.
- Submit this notebook run from top to bottom with outputs, or Python scripts with a PDF report of at most four pages (results and explanations per task). Late: −10% per day, 0 after 5 days.
- Individual work. Name the AI tools you used at the top of your solution.

```
import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader
```

Lab 1

1.1 Tensors

- Write down the expected shapes of five operations (covering reshape, transpose, `@`, `*` and broadcasting, one of them a broadcast that fails), then run them. Explain the failure.
- Show a case where `view` fails and `reshape` works, and explain it with strides.
- Show a computation that `float32` gets wrong and `float64` gets right.
- Convert an array from NumPy to PyTorch and back. Show which conversions share memory, and which dtype each library uses by default.

1.2 Autograd and a gradient check

One-feature logistic regression with binary cross-entropy: $L = -[\,y \log a + (1-y)\log(1-a)\,]$, $a = \sigma(w_1 x_1 + b)$.

- Compute $\partial L/\partial w_1$ and $\partial L/\partial b$ analytically, with autograd and with central finite differences, in `float64`, and check that they match. Reference: for $x_1 = 1.1$, $w_1 = 2.2$, $b = 0$, $y = 1$, $L \approx 0.0852$ and the gradients are ≈ -0.0898 and -0.0817.
- Plot the error of the finite-difference $\partial L/\partial w_1$ (against the analytic value) for $h = 10^{-1}, \ldots, 10^{-12}$ on log–log axes, in `float32` and `float64`. Explain the shape of the curves.
- Call `backward()` on two freshly built losses without resetting `.grad`. Show and explain what happens.

1.3 Linear regression

Default data: $N = 200$, $d = 3$, $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathrm{diag}(1, 4, 0.25))$, $y = \mathbf{w}^{*\top}\mathbf{x} + b^* + 0.1\,\varepsilon$ with $\mathbf{w}^*, b^*$ of your choice.

- Minimise the mean squared error by least squares, and by full-batch gradient descent twice: in NumPy with the gradient derived by hand, and in PyTorch with autograd and `torch.optim.SGD`. Compare the three solutions.
- Find the largest learning rate for which the loss still decreases over 300 steps, experimentally (two significant digits) and from the Hessian. Compare.

1.4 A multilayer network

- Implement `count_parameters` and an MLP as an `nn.Module` built from a list of layer sizes, with ReLU between layers. For sizes `[50, 30, 20, 3]`, check the count (2213) against a sum written by hand.
- Implement a softmax without `torch.softmax` that stays finite for the logits `[1000., 1001., 999.]`.

```python
def count_parameters(model: nn.Module, trainable_only: bool = True) -> int:
    """Number of (trainable) scalar parameters of `model`."""
    raise NotImplementedError
```

Lab 2

2.1 `Dataset` and `DataLoader`

Default data: four classes in the plane, 150 points each, centres $(\pm 2, \pm 2)$, $\mathbf{x} = \boldsymbol{\mu}_c + 0.9\,\boldsymbol{\epsilon}$.

- Write a `Dataset`, split it 60/20/20 with a fixed seed, and build the three `DataLoader`s. Explain why only the training loader is shuffled.

2.2 The training loop

- Implement `compute_loss`, `compute_accuracy` and `train`.
- Train your MLP once with SGD and once with Adam (default: sizes `[2, 30, 20, 4]`, batch size 32, 30 epochs) and plot both validation-loss curves. Pick the run with the lower final validation loss and report its test accuracy.
- Save its `state_dict`, load it into a new model, and show that it gives the same outputs.

```python
def compute_loss(
    model: nn.Module, loader: DataLoader, device: torch.device | str = "cpu"
) -> float:
    """Mean cross-entropy over all predictions in `loader`.

    Logits may be (B, C) with targets (B,), or (B, T, C) with targets (B, T).
    """
    raise NotImplementedError
```

```python
def compute_accuracy(
    model: nn.Module, loader: DataLoader, device: torch.device | str = "cpu"
) -> float:
    """Fraction of correct argmax predictions over all of `loader`."""
    raise NotImplementedError
```

```python
def train(
    model: nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    optimizer: torch.optim.Optimizer,
    num_epochs: int,
    device: torch.device | str = "cpu",
) -> dict[str, list[float]]:
    """Train with cross-entropy.

    Returns {"train_loss": [...], "val_loss": [...]}, one value per epoch.
    """
    raise NotImplementedError
```

2.3 A count-based bigram model

Default text: `the-verdict.txt`. Tokens are characters; build the vocabulary from the whole text and use the last 10% for validation.

- Report training and validation perplexity for $\alpha \in \{0, 0.01, 0.1, 1, 10\}$ (report `inf` where it occurs) and explain both ends. On the default text, expect a validation perplexity of about 12 for $\alpha = 1$.
- Generate 200 characters from the $\alpha = 1$ model.

2.4 A neural bigram model

- Train a table of logits $W \in \mathbb{R}^{V \times V}$ (`nn.Embedding(V, V)`) on the (current, next) character pairs with your `train` (default: batch size 256, Adam with learning rate 0.1, 5 epochs). Compare its validation perplexity with your best count-based model from Task 2.3 and explain the result.
- Compare the loss before training with $\ln V$. Explain the difference, and show that zero-initialised logits give exactly $\ln V$.
- Show with an experiment what plays the role of α (for example more epochs, or weight decay).

2.5 Your own experiment

About one hour of work: one question this list raised, answered with one plot or table and a paragraph. Ideas are in Materials 2.5.